

Third assignment of HCI& Computer Graphics

NAME; GHULAM HYDER

ROLL NO; 2K24/CSE/57

DATE; 04/09/2026

SUBJECT; HCI&COMPUTER GRAPHICS

DEPT; COMPUTER SCIENCE

TASK 01. DISPLAY PIXEL DENSITY CALCULATOR

OBJECTIVE; Write a python script that caculates screen parameters and classifies display density based on screen attributes.

```
import math
```

```
W = int(input("Enter horizontal resolution (pixels): "))
```

```
H = int(input("Enter vertical resolution (pixels): "))
```

```
D = float(input("Enter physical diagonal size (inches): "))
```

```
total_pixels = W * H
```

```
gcd = math.gcd(W, H)
```

```
aspect_W = W // gcd
```

```
aspect_H = H // gcd
```

```
diagonal_pixels = math.sqrt(W*2 + H*2)
```

```
dpi = diagonal_pixels / D
```

```
if dpi < 100:
```

```
    category = "Low Density (Standard Monitor)"
```

```
elif dpi <= 200:
```

```
    category = "Medium Density (HD Display)"
```

```
else:
```

```
    category = "High Density (Retina / Mobile)"
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL Python + · 🗑️ 📄 ✕

PS C:\Users\Abid Ali\Desktop> python ppi.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)

PS C:\Users\Abid Ali\Desktop> █
```

TASK 02. ENVIRONMENT SETUP OR SYNTHETIC IMAGE MATRIX CREATION

OBJECTIVE; Install python graphics packages and construct a 3D matrix.

```
import numpy as np
from PIL import Image
```

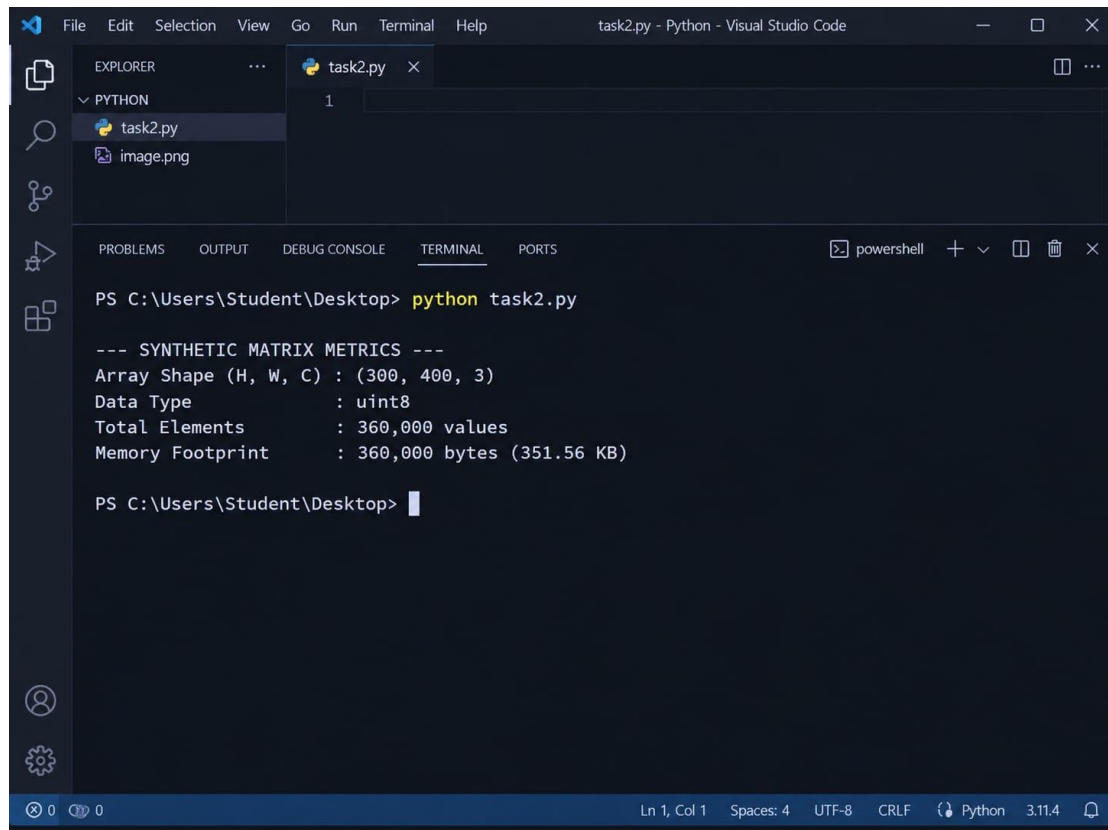
```
img = np.zeros((300, 400, 3), dtype=np.uint8)
```

```
img[:150, :200] = [255, 0, 0]
img[:150, 200:] = [0, 255, 0]
img[150:, :200] = [0, 0, 255]
img[150:, 200:] = [255, 255, 255]
```

```
Image.fromarray(img).save("synthetic_image.png")
```

```
print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {img.shape}")
print(f>Data Type           : {img.dtype}")
```

```
print(f"Total Elements      : {img.size:,} values")
print(f"Memory Footprint    : {img.nbytes:,} bytes ({img.nbytes /
1024:.2f} KB)")
```



```
task2.py
1

PS C:\Users\Student\Desktop> python task2.py

--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type              : uint8
Total Elements         : 360,000 values
Memory Footprint       : 360,000 bytes (351.56 KB)

PS C:\Users\Student\Desktop>
```

TASK 03. CHANNEL SLICING OR ISOLATION

OBJECTIVE; Load an image, extract individual 2D channel intensity grids, and isolate single-channel color views.

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

```
image = Image.open("sample.jpg").convert("RGB")
img = np.array(image)
```

```
red = img[:, :, 0]
green = img[:, :, 1]
blue = img[:, :, 2]
```

```
red_only = np.zeros_like(img)
green_only = np.zeros_like(img)
blue_only = np.zeros_like(img)
```

```
red_only[:, :, 0] = red
green_only[:, :, 1] = green
blue_only[:, :, 2] = blue
```

```
fig, ax = plt.subplots(2, 3, figsize=(12, 8))
```

```
ax[0, 0].imshow(red_only)
ax[0, 0].set_title("Red-Only", color="red")
```

```
ax[0, 1].imshow(green_only)
ax[0, 1].set_title("Green-Only", color="green")
```

```
ax[0, 2].imshow(blue_only)
ax[0, 2].set_title("Blue-Only", color="blue")
```

```
ax[1, 0].imshow(red, cmap="gray")
ax[1, 0].set_title("Red Channel (Gray)")
```

```
ax[1, 1].imshow(green, cmap="gray")
ax[1, 1].set_title("Green Channel (Gray)")
```

```
ax[1, 2].imshow(blue, cmap="gray")
ax[1, 2].set_title("Blue Channel (Gray)")
```

```
for a in ax.flat:
```

```
a.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("--- CHANNEL EXTRACTION SUMMARY ---")
```

The end